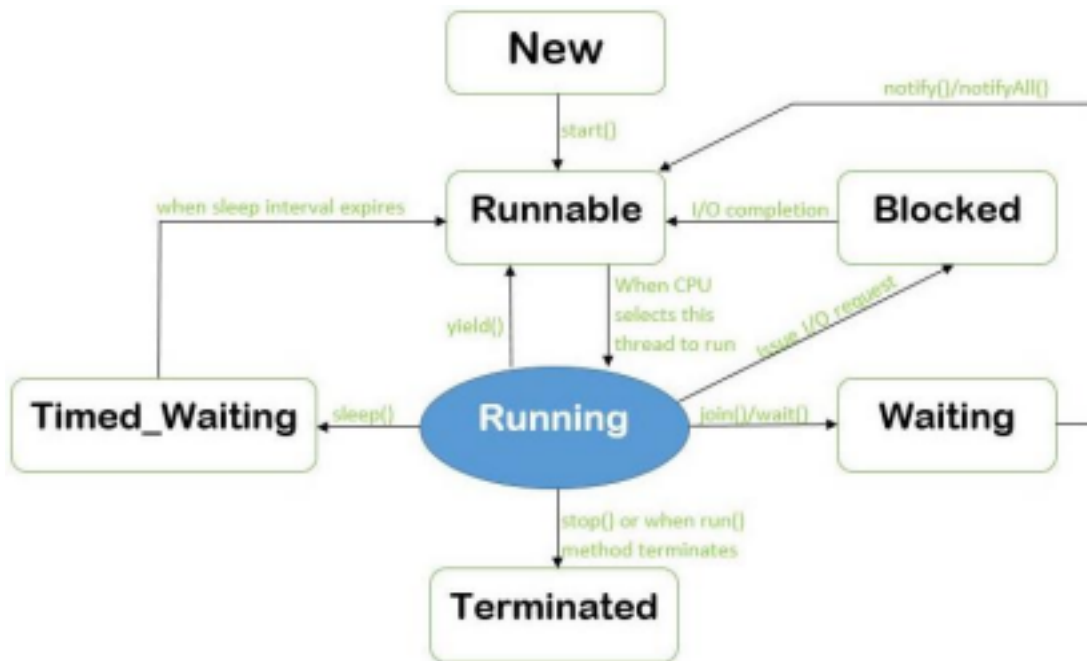


UNIT V – MULTITHREADING, GUI PROGRAMMING & EVENT HANDLING Thread lifecycle, thread creation (Thread class, Runnable interface), Thread synchronization, Inter-thread communication , Java GUI basics: Introduction to AWT and Swing , Layout Managers, Event Delegation Model , Handling GUI Events: ActionListener, Mouse Listener, Building basic forms and interfaces with buttons, labels, text fields, Applications: Chat apps, form-based systems, simulation apps.

5.1 Thread lifecycle:

- In Java, a thread represents a separate path of execution. The lifecycle of a thread is controlled by the Thread class and Runnable interface.
- A thread goes through the following states (defined in Thread.State enum):



1. New

(Created):

- When a thread is created but not yet started, it is in the new state. The thread has not begun execution.
- Thread t = new Thread();
- System.out.println(t.getState()); // Output: NEW

2. Runnable:

- After calling start(), the thread is ready to run but waiting for CPU scheduling.
- Example:
- t.start(); // moves thread to runnable state
- System.out.println(t.getState()); // Output: RUNNABLE

3. Running:

- The thread is in this state when the CPU picks it up from the runnable pool and executes its run() method.
- Only one thread per core can run at a time.

4. Waiting:

- A thread is in the Waiting state when it calls methods like wait(), join(), or LockSupport.park().
- In this state, the thread is waiting indefinitely for another thread to perform a specific action (like notifying it to continue execution, notify() or notifyAll()).

5. Timed Waiting:

- Similar to the Waiting state, a thread enters the Timed Waiting state when it calls methods with a timeout, such as sleep(milliseconds) or wait(milliseconds).
- The thread will remain in this state until the specified time elapses or it receives a notification.
- Example: sleep(1000), join(2000), wait(500).

6. Terminated (Dead):

- A thread enters the Terminated state when it has completed its execution, either by returning from the run() method or by being terminated due to an exception.
- At this point, the thread is no longer alive and cannot be restarted.

5.2 Thread Creation (Thread class, Runnable interface):

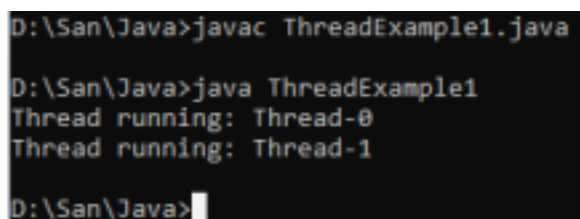
There are two main ways to create threads in Java:

1. By Extending Thread Class:

- Create a class that extends Thread.
- Override the run() method.
- Create an object and call start() (not run() directly!).

Example:

```
class MyThread extends Thread
{
    public void run()
    {
        System.out.println("Thread running: " + Thread.currentThread().getName());
    }
}
public class ThreadExample1
{
    public static void main(String[] args)
    {
        MyThread t1 = new MyThread();
        MyThread t2 = new MyThread();
        t1.start(); // starts thread 1
        t2.start(); // starts thread 2
    }
}
```



```
D:\San\Java>javac ThreadExample1.java
D:\San\Java>java ThreadExample1
Thread running: Thread-0
Thread running: Thread-1
D:\San\Java>
```

2. By Implementing Runnable Interface

- Create a class that implements Runnable.

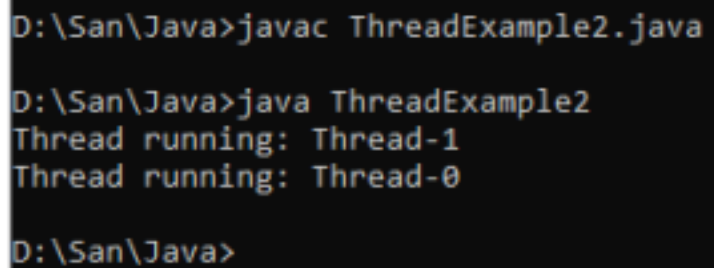
- Override run() method.

- Pass the object of this class to a Thread object and call start().

Example:

```
class MyRunnable implements Runnable
{
    public void run() {
        System.out.println("Thread running: " + Thread.currentThread().getName());
    }
}

public class ThreadExample2
{
    public static void main(String[] args)
    {
        MyRunnable task = new MyRunnable();
        Thread t1 = new Thread(task);
        Thread t2 = new Thread(task);
        t1.start(); // starts thread 1
        t2.start(); // starts thread 2
    }
}
```



```
D:\San\Java>javac ThreadExample2.java

D:\San\Java>java ThreadExample2
Thread running: Thread-1
Thread running: Thread-0

D:\San\Java>
```

Difference between Thread class and Runnable interface

Aspect	Extending Thread	Implementing Runnable
Inheritance	Cannot	
extend any other class	Can	Can still extend another class
Code reusability	Less	
flexible	More	flexible

Usage Simple for small programs Preferred in real-world applications

synchronization:

- When multiple threads access shared resources (like variables, objects, files, databases), race conditions can occur.
- Synchronization ensures that only one thread can access the shared resource at a time that prevents data inconsistency.

Why Synchronization?

Without synchronization:

- Two or more threads may update the same data simultaneously.
- This causes race conditions that lead to unexpected or incorrect results.

1. Synchronized Method

- We can declare a method as synchronized using the synchronized keyword.
- This ensures that only one thread can execute the method at a time for a given object.

2. Synchronized Block

- Instead of synchronizing the entire method, we can synchronize only a part of the code using a synchronized block.
- This improves performance (less locking overhead).

3. Inter-Thread Communication (wait, notify, notifyAll)

Synchronization also allows threads to communicate safely.

- wait() → causes current thread to wait until another thread calls notify().
- notify() → wakes up one waiting thread.
- notifyAll() → wakes up all waiting threads.

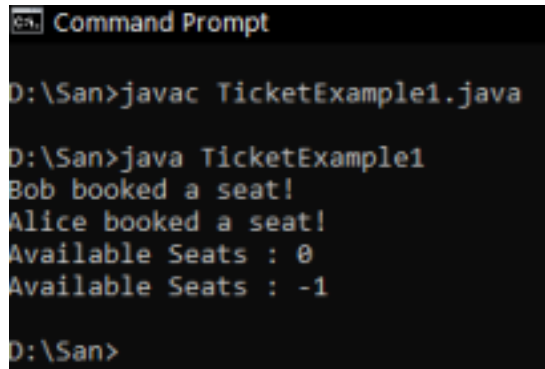
Key Points about Synchronization

- Every object in Java has a monitor lock.
- A synchronized method/block acquires the lock before executing.
- At most one thread can hold the lock at a time.
- Synchronization avoids race conditions but can cause deadlocks if not handled properly.

Example Program - Without Synchronization (Ticket Booking)

```
class TicketBooking {
    private int availableSeats = 1; // only 1 seat
    public void bookTicket(String name)
    {
        if (availableSeats > 0) {
            System.out.println(name + " booked a seat!");
            availableSeats--;
            System.out.println("Available Seats : "+availableSeats);
        } else {
            System.out.println("Sorry, " + name + " no seats available.");
        }
    }
}

public class TicketExample1 {
    public static void main(String[] args) {
        TicketBooking booking = new TicketBooking();
        Thread t1 = new Thread(() -> booking.bookTicket("Alice"));
        Thread t2 = new Thread(() -> booking.bookTicket("Bob"));
        t1.start();
        t2.start();
    }
}
```



```

C:\> Command Prompt

D:\San> javac TicketExample1.java

D:\San> java TicketExample1
Bob booked a seat!
Alice booked a seat!
Available Seats : 0
Available Seats : -1

D:\San>

```

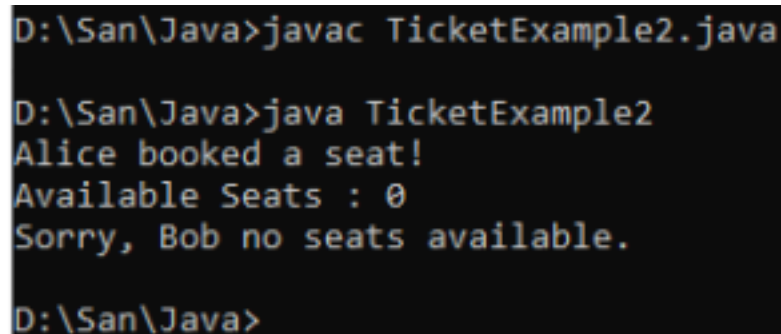
Example Program - With Synchronization (Safe Booking)

```

class TicketBooking
{
    private int availableSeats = 1; // only 1 seat
    public synchronized void bookTicket(String name)
    {
        if (availableSeats > 0)
        {
            System.out.println(name + " booked a seat!");
            availableSeats--;
            System.out.println("Available Seats : "+availableSeats); }
        else {
            System.out.println("Sorry, " + name + " no seats available."); }
    }
}

public class TicketExample2
{
    public static void main(String[] args)
    {
        TicketBooking booking = new TicketBooking();
        Thread t1 = new Thread(() -> booking.bookTicket("Alice"));
        Thread t2 = new Thread(() -> booking.bookTicket("Bob"));
        t1.start();
        t2.start();
    }
}

```



```

D:\San\Java> javac TicketExample2.java

D:\San\Java> java TicketExample2
Alice booked a seat!
Available Seats : 0
Sorry, Bob no seats available.

D:\San\Java>

```

5.4 Inter-thread communication:

When multiple threads work on shared resources, they often need to coordinate. Example: In a restaurant,

- The chef (Producer) prepares food.
- The waiter (Consumer) serves food to customers.
- If the chef keeps cooking without waiting, the kitchen overflows.
- If the waiter serves without food, customers get angry.

They must coordinate. That's what inter-thread communication is. Java provides three important methods (in Object class) for communication: **1.**

wait()

- Causes the current thread to release the lock and go into a waiting state.
- It will wait until another thread notifies it.
- Must be called inside synchronized.

2. notify()

- Wakes up one thread that is waiting on the object's lock.
- The chosen thread will move from waiting → runnable state.

3. notifyAll()

- Wakes up all threads waiting on the object's lock.
- But only one will get the lock and proceed.

Flow of Inter Thread Communication:

- Thread acquires lock (synchronized).
- Thread checks a condition.
- If condition not met → calls wait() → releases lock → goes into waiting pool.
- Another thread does its work and calls notify() or notifyAll().
- Waiting thread(s) move back to runnable and fight for CPU time.

Example 1: Simple Explanation with Bank Deposit/Withdraw

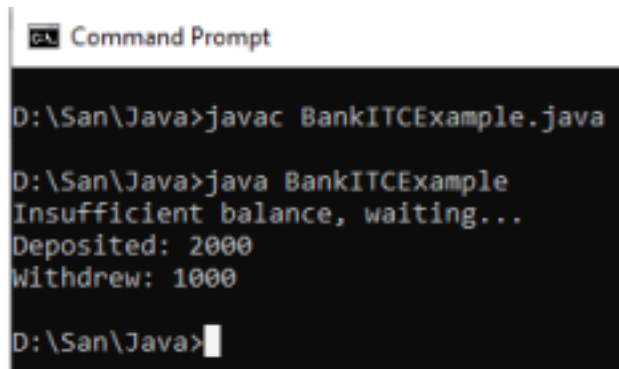
```
class BankAccount
{
    private int balance = 0;
    public synchronized void deposit(int amount)
    {
        balance += amount;
        System.out.println("Deposited: " + amount);
        notify(); // wake up waiting withdraw thread
    }
    public synchronized void withdraw(int amount)
    {
        while (balance < amount)
        { // wait until enough balance
            System.out.println("Insufficient balance, waiting...");
            try
            {
```

```

        wait();
    }
    catch (InterruptedException e) {}
}
balance -= amount;
System.out.println("Withdrew: " + amount);
}
}
public class BankITCExample
{
    public static void main(String[] args)
    {
        BankAccount account = new BankAccount();
        Thread withdrawThread = new Thread(() -> account.withdraw(1000));
        Thread depositThread = new Thread(() -> account.deposit(2000));
        withdrawThread.start();
        depositThread.start();
    }
}

```

Output:



```

Command Prompt

D:\San\Java>javac BankITCExample.java

D:\San\Java>java BankITCExample
Insufficient balance, waiting...
Deposited: 2000
Withdrew: 1000

D:\San\Java>

```

5.5 Introduction to AWT and Swing:

5.5.1. AWT:

- AWT (Abstract Window Toolkit) is a part of Java's standard library used to create Graphical User Interfaces (GUI).
- It is platform-dependent, meaning it relies on the operating system's native GUI components.
- It belongs to the java.awt package. Provides GUI components like Button, Label, TextField, TextArea, Checkbox, List, Choice, Frame, Panel.
- Uses peer classes (native resources of the operating system).
- Event-driven programming model (actions like clicking a button trigger events).

AWT Hierarchy

- At the top level, AWT components are derived from:
Object → Component → Container → Window → Frame/Dialog
- Component: Basic buildingblock (Button, Label, TextField).

- Container: Can hold other components (Panel, Frame).
- Window: Top-level window (cannot be contained inside another container).
- Frame: Main window with title bar, resize, minimize/maximize options. □
- Dialog: Popup window for interaction.

Commonly Used AWT Classes

- ✓ Label – displays text.
- ✓ Button – clickable button.
- ✓ TextField – single-line text input.
- ✓ TextArea – multi-line text input.
- ✓ Checkbox – selectable checkbox.
- ✓ CheckboxGroup – radio buttons.
- ✓ Choice – dropdown menu.
- ✓ List – list of items.
- ✓ Frame – top-level window.

5.5.2. Swing:

- Swing is a part of Java Foundation Classes (JFC) that is used to create Graphical User Interface (GUI) applications.
- It is built on top of AWT (Abstract Window Toolkit) but is more powerful, flexible, and lightweight.
- Swing provides a rich set of components like buttons, labels, text fields, tables, trees, and more.
- All Swing classes are defined in the package: javax.swing.*

Difference Between AWT and Swing:

Feature	AWT	Swing
Package	java.awt	javax.swing
Component Type	Heavyweight (depends on OS)	Lightweight (pure Java)
Speed	Faster (direct OS calls)	Slightly slower (runs above AWT)
Component Variety	Limited (basic controls only)	Rich set (tables, trees, sliders, tooltips, tabbed panes, etc.)

Feature	AWT	Swing
Architecture	Simpler	MVC-based

Extensibility	Less flexible	(Model-View-Controller) More flexible & customizable Uses same event model, more consistent JButton, JLabel, JTextField, JFrame
Event Handling	Delegation event model (introduced later)	
Example	Button, Label, TextField, Frame	

Common Swing Components:

Component	Description
JFrame	Top-level window for GUI applications
JLabel	Displays a text or image
JButton	A clickable button
JTextField	Allows user to enter a single line of text
JTextArea	Multi-line text input
JCheckBox	Checkbox for true/false input
JRadioButton	Radio button for single-choice selection
JList	Displays a list of items
JTable	Displays tabular data
JMenu, JMenuItem	Menus and menu items

1. JFrame:

- JFrame is a class in the javax.swing package that represents a top-level window with a title bar, border, and buttons (close, minimize, maximize).
- It is the main container in a Swing application where we place other GUI components like buttons, labels, text fields, menus, panels, etc.
- Can hold menus (JMenuBar), panels (JPanel), and many Swing components.
- Often used as the main window in desktop applications.

Useful JFrame Methods:

Method	Description
setTitle(String title)	Sets the window title

setSize(int w, int h) setLocation(int x, int y)	Sets width & height
	Sets window position

Method	Description
setLocationRelativeTo(nu ll) setResizable(boolean flag)	Centers the window Allows/disallows resizing
setDefaultCloseOperation(int op) add(Component c)	Defines behavior when closing window Adds a component
setLayout(LayoutManager m) setVisible(true)	Sets layout manager Makes window visible

```
import javax.swing.*;
public class SimpleJFrame
{
    public static void main(String[] args)
    {
        JFrame frame = new JFrame("My First JFrame");
        frame.setSize(300, 150); // set size
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // close the app
        frame.setLocationRelativeTo(null); // center the frame on screen
        frame.setVisible(true); // make it visible
    }
}
```



2. JLabel:

- JLabel is a GUI component in Swing used to display a short string of text, an image, or both.
- It is non-editable (unlike JTextField), meaning users cannot modify its content directly.
- Supports alignment (left, center, right).

□ Often used for labels beside text fields, buttons, or images.

Common Constructors:

`JLabel()` // empty label

`JLabel(String text)` // label with text

`JLabel(Icon image)` // label with image

`JLabel(String text, int alignment)` // text with alignment

`JLabel(String text, Icon icon, int alignment)` // text + image + alignment

Common Methods:

Method	Description
<code>setText(String text)</code>	Sets the text
<code>getText()</code>	Returns the text
<code>setIcon(Icon icon)</code>	Sets an image
<code>setHorizontalAlignment(int align)</code>	Aligns LEFT, CENTER, RIGHT
<code>setForeground(Color c)</code>	Sets text color
<code>setFont(Font f)</code>	Sets font style and size

```
import javax.swing.*;
public class JLabelExample
{
    public static void main(String[] args)
    {
        JFrame frame = new JFrame("JLabel Example");
        JLabel label = new JLabel("Hello, I am a JLabel!"); //Create a label with text
        label.setBounds(100, 50, 200, 30);
        frame.add(label); // Add label to frame
        frame.setSize(300, 150); // Frame settings
        frame.setLayout(null);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```



3. JButton:

□ JButton is a Swing component that represents a push button.

- When the user clicks it, an action event is triggered.
- It can display text, icon (image), or both.
- It often used for submitting forms, triggering actions, opening dialogs, etc.

Constructors:

.JButton() // Empty button

JButton(String text) // Button with text

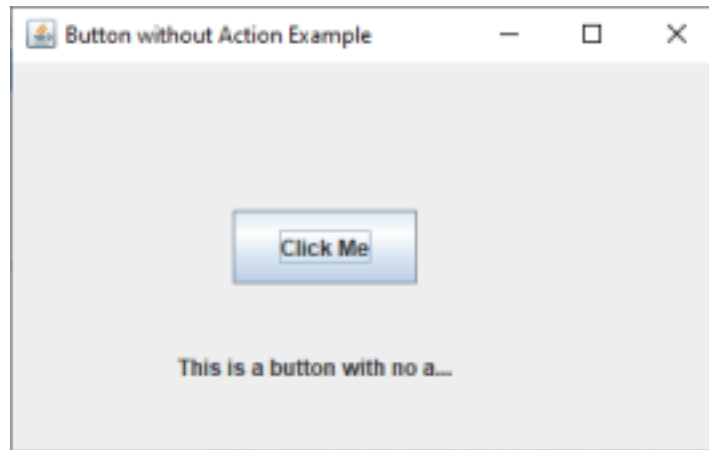
JButton(Icon image) // Button with image

JButton(String text, Icon image) // Button with text + image

Useful Methods:

Method	Description
setText(String text)	Sets button label
getText()	Gets current text
setIcon(Icon icon)	Sets an image icon
setEnabled(boolean b)	Enables/disables
addActionListener(ActionListener l)	button Adds click listener
setToolTipText(String text)	Shows tooltip on hover

```
import javax.swing.*;
public class JButtonNoAction
{
    public static void main(String[] args)
    {
        JFrame frame = new JFrame("Button without Action Example");
        JButton button = new JButton("Click Me"); // Create a button
        button.setBounds(120, 80, 100, 40);
        JLabel label = new JLabel("This is a button with no action.");
        label.setBounds(90, 150, 150, 30);
        frame.add(button); // Add components
        frame.add(label);
        frame.setSize(400, 250); // Frame settings
        frame.setLayout(null);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```



4. JTextField:

- JTextField is a Swing component used to enter and edit a single line of text. □ Commonly used for inputs like username, email, numbers, search boxes, etc. □ It allows user input (editable text).
- It supports event handling (e.g., pressing Enter triggers an `ActionEvent`).
- We can set the default text, columns, font, colors, etc.

Constructors:

`JTextField()` // empty text field
`JTextField(String text)` // with initial text
`JTextField(int columns)` // sets width (in characters)
`JTextField(String text, int cols)` // with text and width

Useful Methods:

Method	Description
<code>setText(String text)</code>	Sets the text
<code>getText()</code>	Returns the current text
<code>setEditable(boolean b)</code>	Makes field editable or
<code>setColumns(int n)</code>	read-only Sets number of
<code>setFont(Font f)</code>	columns
<code>setForeground(Color c)</code>	Changes font style
<code>setBackground(Color c)</code>	Changes text color
<code>addActionListener(ActionListener l)</code>	Changes background color
	Triggers event on Enter key

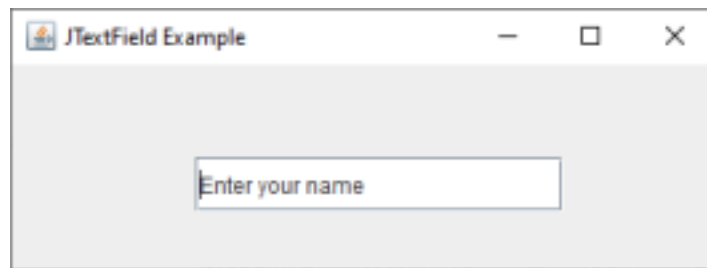
```

import javax.swing.*;
public class JTextFieldExample
{
    public static void main(String[] args)
  
```

```

{
JFrame frame = new JFrame("JTextField Example");
JTextField textField = new JTextField("Enter your name");
textField.setBounds(100, 50, 200, 30);
frame.add(textField);
frame.setSize(400, 150);
frame.setLayout(null);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setVisible(true);
}
}

```



5.6 Layout Managers:

- In Java AWT and Swing, a Layout Manager is an object that controls the size, position, and arrangement of components (like buttons, labels, text fields) inside a container (like Frame, Panel, JFrame, or JPanel).
- Instead of manually setting positions using `setBounds(x, y, width, height)`, layout managers automatically adjust component placement based on container size, screen resolution, and resizing.
- Defined in the package: `java.awt`
- Default:
 - ✓ For Frame/JFrame → `BorderLayout`.
 - ✓ For Panel/JPanel → `FlowLayout`

Complete List of Layout Managers in Java

- | | | | |
|---------------------------|-----------------|----|---------------------------------------|
| 1. FlowLayout | 2. BorderLayout | 3. | 7. GroupLayout (Swing only, Java SE |
| GridLayout | 4. CardLayout | 5. | 6+) 8. SpringLayout (Swing only) |
| GridBagLayout | | | 9. Null Layout (absolute positioning) |
| 6. BoxLayout (Swing only) | | | |

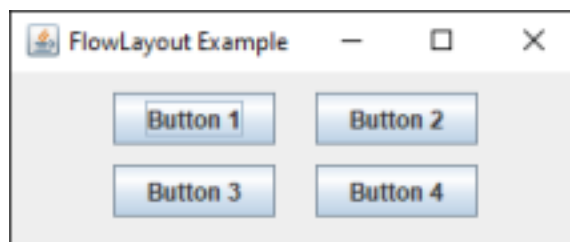
- ✓ For simple apps → FlowLayout, BorderLayout, GridLayout.
- ✓ For professional apps → GridBagLayout, BoxLayout, GroupLayout. ✓ For special cases → CardLayout (wizards), SpringLayout (constraints), Null Layout (games).

1. FlowLayout:

- Arranges components in a row, left to right.

- When space is insufficient, moves components to the next line.
- Alignments: LEFT, CENTER (default), RIGHT.

```
import java.awt.*;
import javax.swing.*;
public class FlowLayoutExample {
    public static void main(String[] args) {
        JFrame f = new JFrame("FlowLayout Example");
        // align=center, hgap=20, vgap=10
        f.setLayout(new FlowLayout(FlowLayout.CENTER, 20, 10));
        f.add(new JButton("Button 1"));
        f.add(new JButton("Button 2"));
        f.add(new JButton("Button 3"));
        f.add(new JButton("Button 4"));
        f.setSize(200, 200);
        f.setVisible(true);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); }
}
```



2. BorderLayout:

- Divides the container into 5 regions:
NORTH, SOUTH, EAST, WEST, CENTER
- Only one component per region.
- CENTER expands to fill unused space.

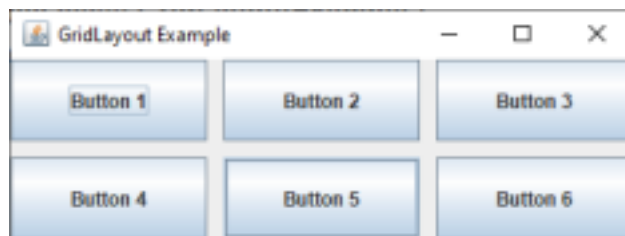
```
import java.awt.*;
import javax.swing.*;
public class BorderLayoutExample
{
    public static void main(String[] args) {
        JFrame f = new JFrame("BorderLayout Example");
        f.setLayout(new BorderLayout());
        f.add(new JButton("North"), BorderLayout.NORTH);
        f.add(new JButton("South"), BorderLayout.SOUTH);
        f.add(new JButton("East"), BorderLayout.EAST);
        f.add(new JButton("West"), BorderLayout.WEST);
        f.add(new JButton("Center"), BorderLayout.CENTER);
        f.setSize(250, 250);
        f.setVisible(true);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); }
}
```



3. GridLayout:

- Divides container into a grid of rows and columns.
- All cells are of equal size.

```
import java.awt.*;
import javax.swing.*;
public class GridLayoutExample {
    public static void main(String[] args) {
        JFrame f = new JFrame("GridLayout Example");
        f.setLayout(new GridLayout(2, 3, 10, 10)); // 2 rows, 3 columns, gaps for
        (int i = 1; i <= 6; i++) {
            f.add(new JButton("Button " + i));
        }
        f.setSize(400, 150);
        f.setVisible(true);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); }
}
```



4. CardLayout

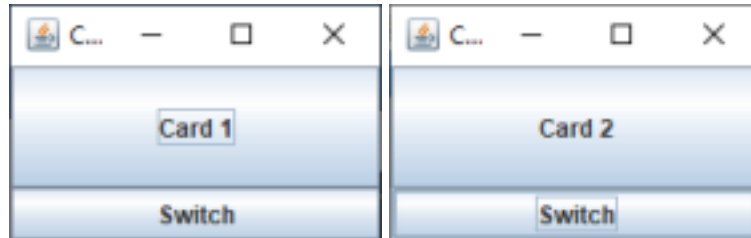
- Stacks components like a deck of cards.
- Only one component is visible at a time.
- Useful for wizards or tabbed UI.

```
import java.awt.*;
import javax.swing.*;
import java.awt.event.*;
public class CardLayoutExample
{
    public static void main(String[] args)
    {
        JFrame f = new JFrame("CardLayout Example");
        CardLayout card = new CardLayout();
        JPanel panel = new JPanel(card);
        panel.add(new JButton("Card 1"), "card1");
        panel.add(new JButton("Card 2"), "card2");
        JButton switchBtn = new JButton("Switch");
```

```

switchBtn.addActionListener(e -> card.next(panel));
f.add(panel, BorderLayout.CENTER);
f.add(switchBtn, BorderLayout.SOUTH);
f.setSize(200, 125);
f.setVisible(true);
f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); }
}

```



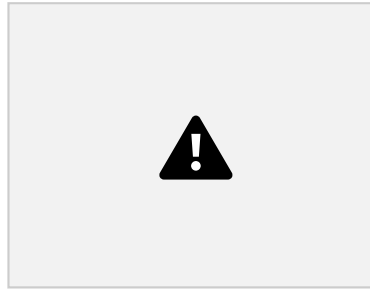
5. GridBagLayout:

- The most flexible and complex layout manager.
- Places components in a grid, but unlike GridLayout, cells can have different sizes, and components can span multiple rows/columns.
- Controlled using GridBagConstraints.

```

import java.awt.*;
import javax.swing.*;
public class GridBagLayoutExample
{
    public static void main(String[] args)
    {
        JFrame f = new JFrame("GridBagLayout Example");
        f.setLayout(new GridBagLayout());
        GridBagConstraints gbc = new GridBagConstraints();
        JButton b1 = new JButton("Button 1");
        JButton b2 = new JButton("Button 2");
        JButton b3 = new JButton("Button 3");
        gbc.gridx = 0; gbc.gridy = 0;
        f.add(b1, gbc);
        gbc.gridx = 1; gbc.gridy = 0;
        f.add(b2, gbc);
        gbc.gridx = 0; gbc.gridy = 1; gbc.gridwidth = 2;
        f.add(b3, gbc);
        f.setSize(200, 150);
        f.setVisible(true);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}

```

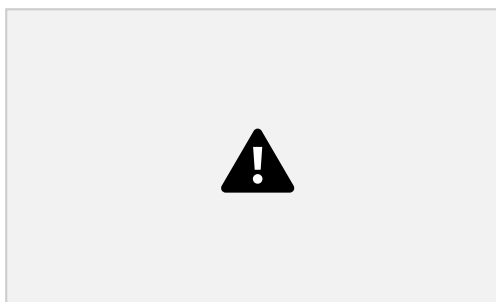
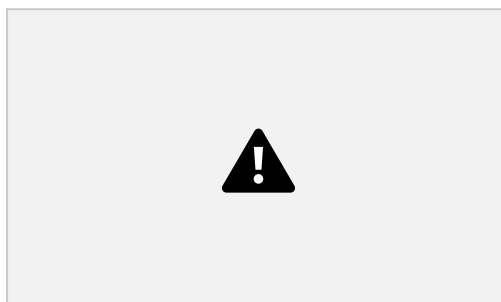


6. BorderLayout (Swing only)

- Arranges components either vertically (Y-axis) or horizontally (X-axis) in a single row or column.
- Unlike FlowLayout, components do not wrap to the next line.
- Belongs to javax.swing package.

```
import javax.swing.*;
import java.awt.*;
public class BorderLayoutExample
{
    public static void main(String[] args)
    {
        JFrame f = new JFrame("BorderLayout Example");
        JPanel panel = new JPanel();
        panel.setLayout(new BorderLayout(panel, BorderLayout.Y_AXIS)); //
            vertical alignment
        panel.add(new JButton("Button 1"));
        panel.add(new JButton("Button 2"));
        panel.add(new JButton("Button 3"));
        f.add(panel);
        f.setSize(250, 150);
        f.setVisible(true);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```

Y-axis: X-axis:



7.

GridLayout (Swing only, Java SE 6+)

- Introduced in Java SE 6.
- Used mainly by GUI builders (like NetBeans).
- Allows placement of components in hierarchical groups, both horizontally and vertically.

- Provides very precise alignment control.

8. **SpringLayout (Swing only):**

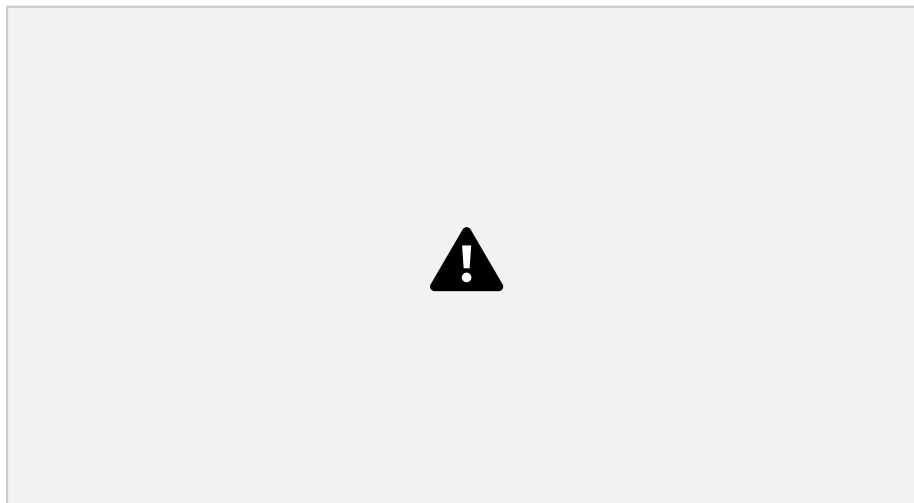
- Very flexible but low-level.
- Defines relationships (springs and constraints) between edges of components.
- Example: "Button A should always be 20 pixels to the right of Button B". □
- Rarely used manually → mostly used by IDE GUI builders.

9. **Null Layout (absolute positioning):**

- Not technically a layout manager (it means no layout manager). □ We can use `setLayout(null)` and place components manually with `setBounds(x, y, width, height)`. Not portable across screen resolutions.
- Not resizable. But sometimes used for custom UIs and games.

5.7 Event Delegation Model:

- The Event Delegation Model (EDM) is a mechanism in Java (introduced with Java 1.1) to handle events in GUI programming (AWT & Swing).
- Instead of having every component handle its own events, the event delegation model separates the event source from the event listener.
- This makes event handling cleaner, flexible, and more object-oriented.



Components of the Event Delegation Model

1. Event Source

- The GUI component that generates events when a user interacts with it.
- Example: Button, TextField, Checkbox.
- The event source is responsible for:
 - ✓ Creating an Event Object when an action happens.
 - ✓ Invoking the registered listener's callback method.

2. Event Object

- When an action occurs, the Event Source creates an Event Object. □ The Event Object carries information about the event (e.g., mouse click

position, button label, key pressed.

□ This Event Object is then passed to the Listener Object.

□ Example classes:

✓ ActionEvent → Button click

✓ MouseEvent → Mouse operations

✓ KeyEvent → Keyboard operations

✓ WindowEvent → Closing/minimizing a window

3. Listener Interface

□ Defines the contract for handling a specific type of event.

□ Example interfaces: ActionListener, MouseListener, KeyListener.

□ Contains one or more abstract methods (like actionPerformed, mouseClicked) that must be implemented.

□ Example interfaces:

✓ ActionListener → actionPerformed(ActionEvent e)

✓ MouseListener → mouseClicked(MouseEvent e)

✓ KeyListener → keyPressed(KeyEvent e)

4. Listener Object

□ A concrete class that implements the Listener Interface.

□ It contains the actual callback methods that define what should happen when an event occurs.

□ Example:

```
class MyHandler implements ActionListener {  
    public void actionPerformed(ActionEvent e) {  
        System.out.println("Button Clicked!");  
    }  
}
```

Working of Event Delegation Model

□ User performs an action (click, press a key, move mouse).

□ Event Source generates an Event Object.

□ Event Source notifies all registered Event Listeners.

□ The listener's callback method executes to handle the event.

Example Program:

```
import javax.swing.*;  
import java.awt.event.*;  
public class SwingEDM  
{  
    public static void main(String[] args) {
```

```

JFrame f = new JFrame("Swing Event Example");
JButton b = new JButton("Click");
b.setBounds(100, 100, 100, 40);
// Registering an ActionListener
b.addActionListener(e -> JOptionPane.showMessageDialog(f, "Button Clicked!"));
f.add(b);
f.setSize(300, 200);
f.setLayout(null);
f.setVisible(true);
f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
}
}

```

Output:



ActionListener:

- ActionListener is an interface in the java.awt.event package.
- It is used to handle action events such as:
 - ✓ Clicking a button (JButton)
 - ✓ Choosing a menu item (JMenuItem)
 - ✓ Pressing Enter in a text field (JTextField)
- In short, whenever the user performs an action event, the ActionListener's actionPerformed() method is executed.

Syntax:

```

import java.awt.event.ActionListener;
import java.awt.event.ActionEvent;

public interface ActionListener extends EventListener
{
    void actionPerformed(ActionEvent e);
}

```

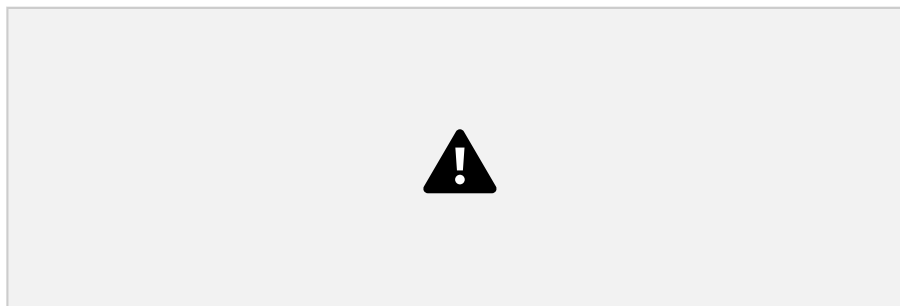
Steps to use ActionListener:

- Create a GUI component (e.g., JButton).
- Implement ActionListener interface or use an anonymous class/lambda.
- Override the actionPerformed(ActionEvent e) method.

□ Register the listener using addActionListener() method.

Example:

```
import javax.swing.*;
import java.awt.event.*;
public class ActionListenerExample implements ActionListener
{
    JButton button;
    ActionListenerExample()
    {
        JFrame frame = new JFrame("ActionListener Example");
        button = new JButton("Click Me");
        button.setBounds(50, 50, 120, 40);
        button.addActionListener(this); // Register ActionListener
        frame.add(button);
        frame.setSize(250, 150);
        frame.setLayout(null);
        frame.setVisible(true);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); }
    // This method is called when the button is clicked
    public void actionPerformed(ActionEvent e)
    {
        JOptionPane.showMessageDialog(null, "Button Clicked!"); }
    public static void main(String[] args)
    {
        new ActionListenerExample();
    }
}
```



5.9 Mouse Listener:

□ MouseListener is an interface in java.awt.event package.

□ It is used to handle mouse events like:

- ✓ Mouse clicked
- ✓ Mouse pressed
- ✓ Mouse released
- ✓ Mouse entered (cursor enters a component area)
- ✓ Mouse exited (cursor leaves a component area)

Syntax:

```
import java.awt.event.MouseListener;
```

```
import java.awt.event.MouseEvent;

public interface MouseListener extends EventListener
{
    void mouseClicked(MouseEvent e);
    void mousePressed(MouseEvent e);
    void mouseReleased(MouseEvent e);
    void mouseEntered(MouseEvent e);
    void mouseExited(MouseEvent e);
}
```

Steps to Use MouseListener:

- ✓ Create a component (e.g., JButton, JPanel, or JLabel).
- ✓ Implement MouseListener interface.
- ✓ Override all five methods.
- ✓ Register the listener using addMouseListener().

```
import javax.swing.*;
import java.awt.event.*;

public class MouseListenerDialogExample extends JFrame implements MouseListener
{
    JButton button;

    MouseListenerDialogExample()
    {
        button = new JButton("Hover or Click Me");
        button.setBounds(100, 100, 200, 50);
        // Register MouseListener
        button.addMouseListener(this);
        add(button);
        setSize(400, 300);
        setLayout(null);
        setVisible(true);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
    }
    // Implement all methods with dialog boxes
    public void mouseClicked(MouseEvent e)
    {
        JOptionPane.showMessageDialog(this, "Mouse Clicked!");
    }
    public void mousePressed(MouseEvent e)
    {
        JOptionPane.showMessageDialog(this, "Mouse Pressed!");
    }
    public void mouseReleased(MouseEvent e)
    {
        JOptionPane.showMessageDialog(this, "Mouse Released!");
    }
    public void mouseEntered(MouseEvent e)
    {
        JOptionPane.showMessageDialog(this, "Mouse Entered!");
    }
}
```

```

public void mouseExited(MouseEvent e)
{
JOptionPane.showMessageDialog(this, "Mouse Exited!");
}
public static void main(String[] args)
{
new MouseListenerDialogExample();
}
}

```



5.10 Building basic forms and interfaces with buttons, labels, text fields:

```

import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
public class SimpleForm
{
    public static void main(String[] args)
    {
        JFrame frame = new JFrame("Simple Form"); // Create a frame
        frame.setSize(300, 150);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLayout(new FlowLayout()); // Using FlowLayout
        JLabel label = new JLabel("Enter your name:"); // Create components
        JTextField textField = new JTextField(15);
        JButton button = new JButton("Submit");
        button.addActionListener(new ActionListener() // Add action listener for button {
        public void actionPerformed(ActionEvent e)
        {
            String name = textField.getText();
            JOptionPane.showMessageDialog(frame, "Hello, " + name + "!"); }
        });
        frame.add(label); // Add components to frame
        frame.add(textField);
        frame.add(button);
        frame.setVisible(true); // Make frame visible
    }
}

```

